Lecture 8:

Elliptic Curve Crypto

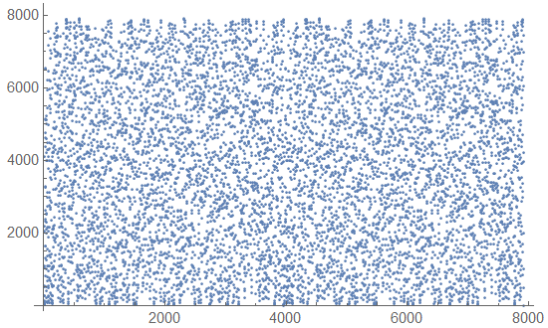
A (minimal) introduction

Giuseppe Bianchi

Why bothering with ECC?

- **Computational security = Hard problems**
  - ⇨ *Easy in one way*
  - ⇨ *Hard in the other way*
- **Example: discrete logarithm**
  - ⇨ *Z*(p), p>2 prime, group order q*
    - → x → g^x mod p easy
    - → g^x → x mod p hard

2^x mod 7919




```
In[31]:= p = 59;
         l = PowerMod[2, Range[1, p - 1], p]
         ListPlot[l]

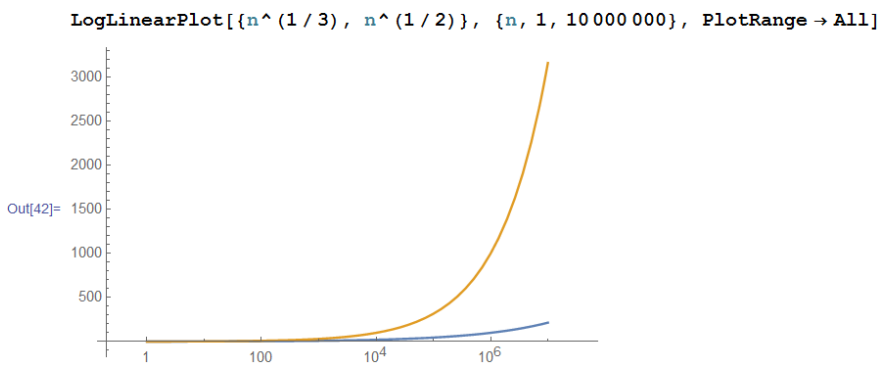
Out[32]= {2, 4, 8, 16, 32, 5, 10, 20, 40, 21, 42, 25, 50, 41, 23, 46, 33, 7, 14, 28, 56, 53, 47, 35, 11, 22, 44, 29, 58,
          57, 55, 51, 43, 27, 54, 49, 39, 19, 38, 17, 34, 9, 18, 36, 13, 26, 52, 45, 31, 3, 6, 12, 24, 48, 37, 15, 30, 1}
```

⇨ *Not restricted to Z*(p), can be* **any** *cyclic group Gp*

Why bothering with ECC?

➔**Same problem (e.g. Dlog, factoring), different "hardness" in different groups!**

- ⇨*Dlog in Z*(p): exp(O($n^{1/3}$))*
- ⇨*Dlog in EC: exp(O($n^{1/2}$))*
- ⇨*Dlog in (Z(p),+)): poly!*



Symm key equiv	*modulus size*	*Elliptic Curve*
80 bits	*1024 bits*	*163 bits*
128 bits	*3072 bits*	*283 bits*
256 bits	***15360** bits*	*571 bits*

EC scales well (so far) with increased security parameter!!

Obvious long term deployment choice (now clear why so many new EC ciphers in TLS)!

Elliptic curves

➔**They are NOT ellipses!**

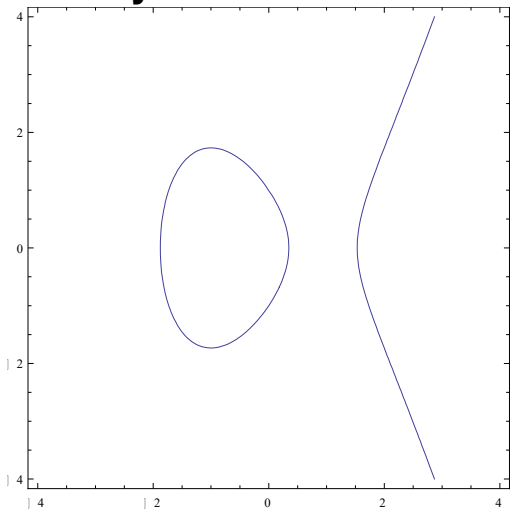
⇨*They are special cubic curves*

⇨*Named from elliptic integrals*

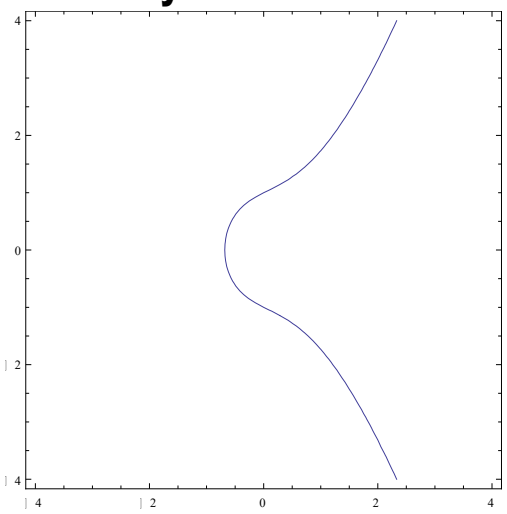
➔**General (Weierstrass) expression**

$$y^2 = x^3 + ax + b \quad where \quad 4a^3 + 27b^2 \neq 0$$

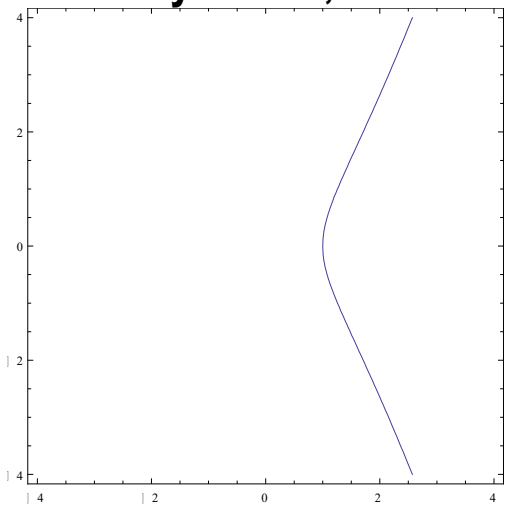
$y^2 = x^3-3x+1$



$y^2 = x^3+x+1$



$y^2 = x^3-1,$



We need an operation

➔We want an operation

⇨*call it + or ○*

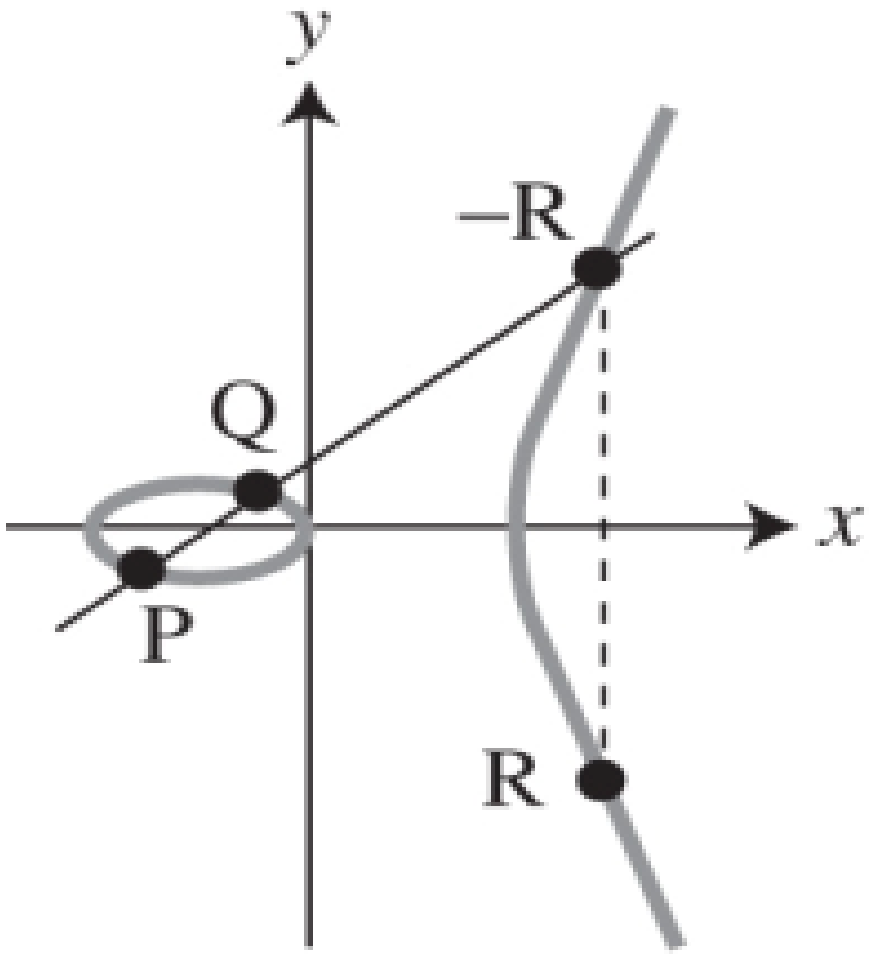
➔historically called +, though someone today may prefer mult. notation

➔So that P+Q=R ➔ R BELONGS to the curve

➔AND the operation gives algebraic group structure

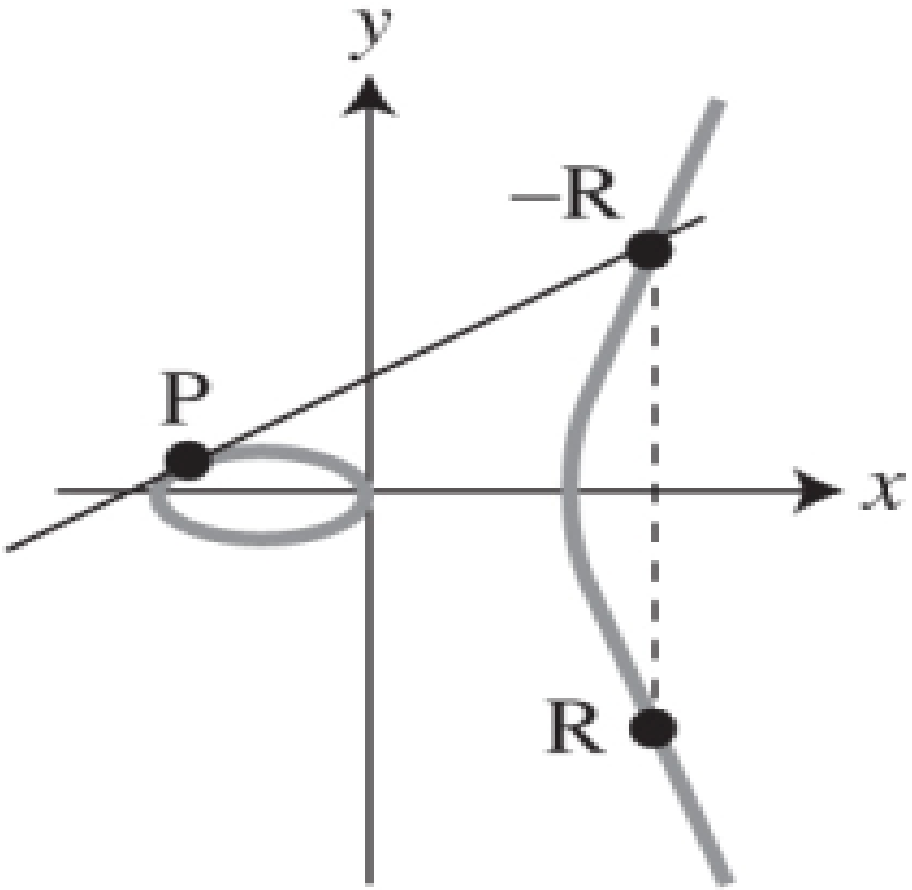
Elliptic points addition P+Q

(geometric)



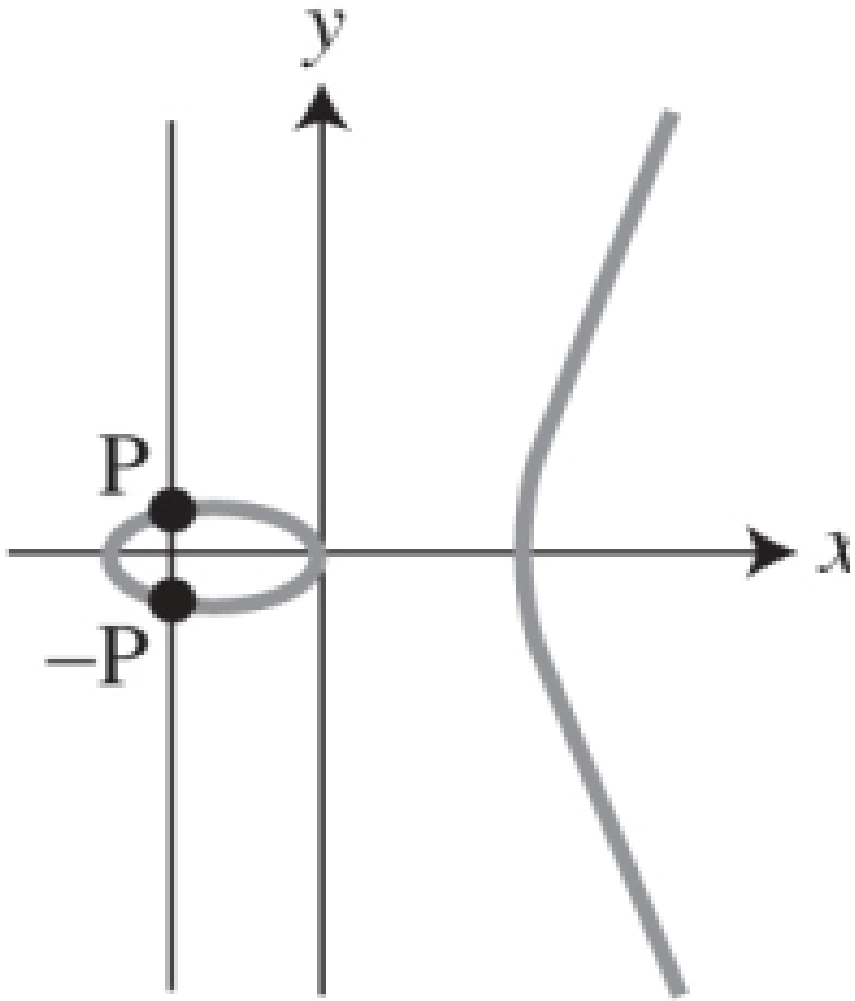
Elliptic points addition P+P

(geometric)



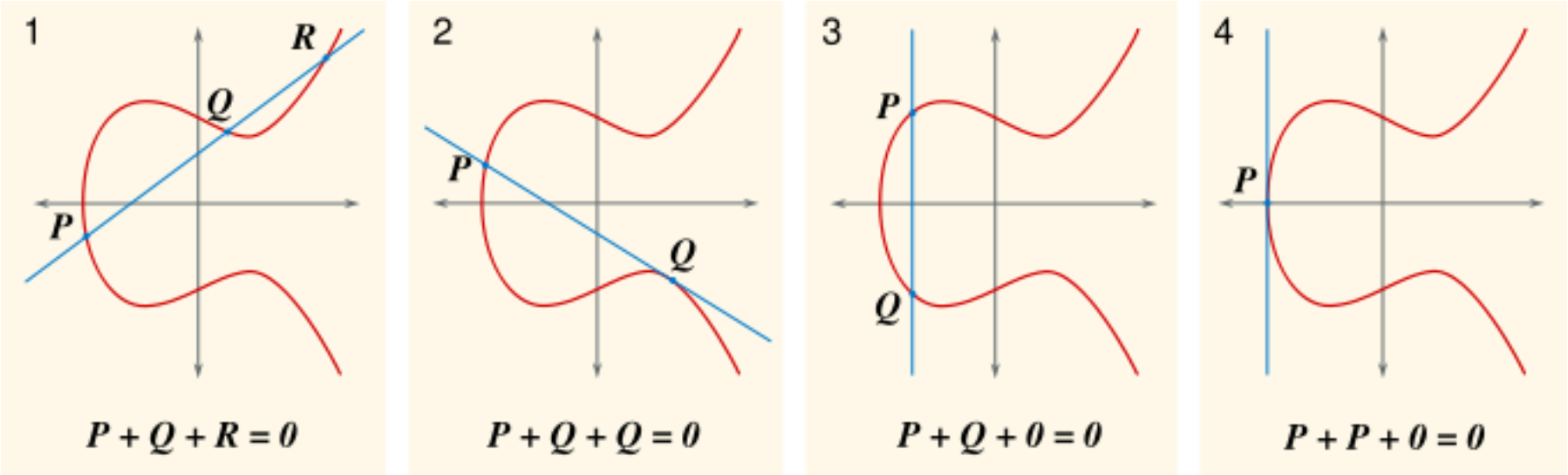
Elliptic points addition P-P
(geometric)

- **"complete" the curve by adding point O=-O to infinity**
- **P+O=P**
  - *O is (additive) zero for the group of EC points*
  - *Now clear why the "inversion" ☺*



General rule

➔**If three points of an elliptic curve lay on a same line, their sum is 0**



Algebraic expressions

➔ **Can be derived from geometric interpretation**

$$P = (x_1, y_1)$$

$$Q = (x_2, y_2)$$

$$R = P + Q = (x_3, y_3)$$

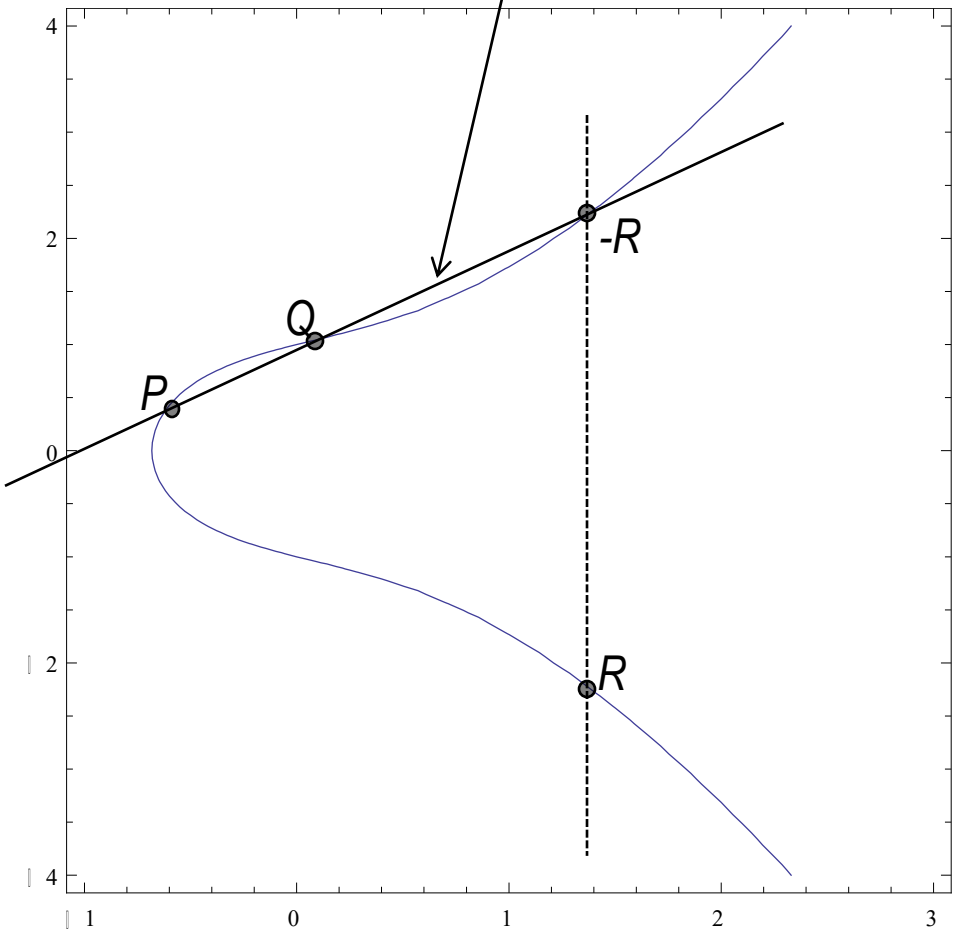
$$x_3 = \lambda^2 - x_1 - x_2$$

$$y_3 = \lambda(x_1 - x_3) - y_1$$

$$\lambda = \begin{cases} \dfrac{y_2 - y_1}{x_2 - x_1} & P \neq Q \\ \dfrac{3x_1^2 + a}{2y_1} & P = Q \end{cases}$$

P≠Q sketch

$$y = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1) = \lambda(x - x_1) + y_1$$



$$y^2 = x^3 + ax + b$$

$$\left(\lambda(x - x_1) + y_1\right)^2 = x^3 + ax + b$$

$$0 = x^3 - \lambda^2 x^2 + \ldots$$

recall :

$$(x - x_1)(x - x_2)(x - x_3) =$$

$$= x^3 - (x_1 + x_2 + x_3)x^2 + \ldots$$

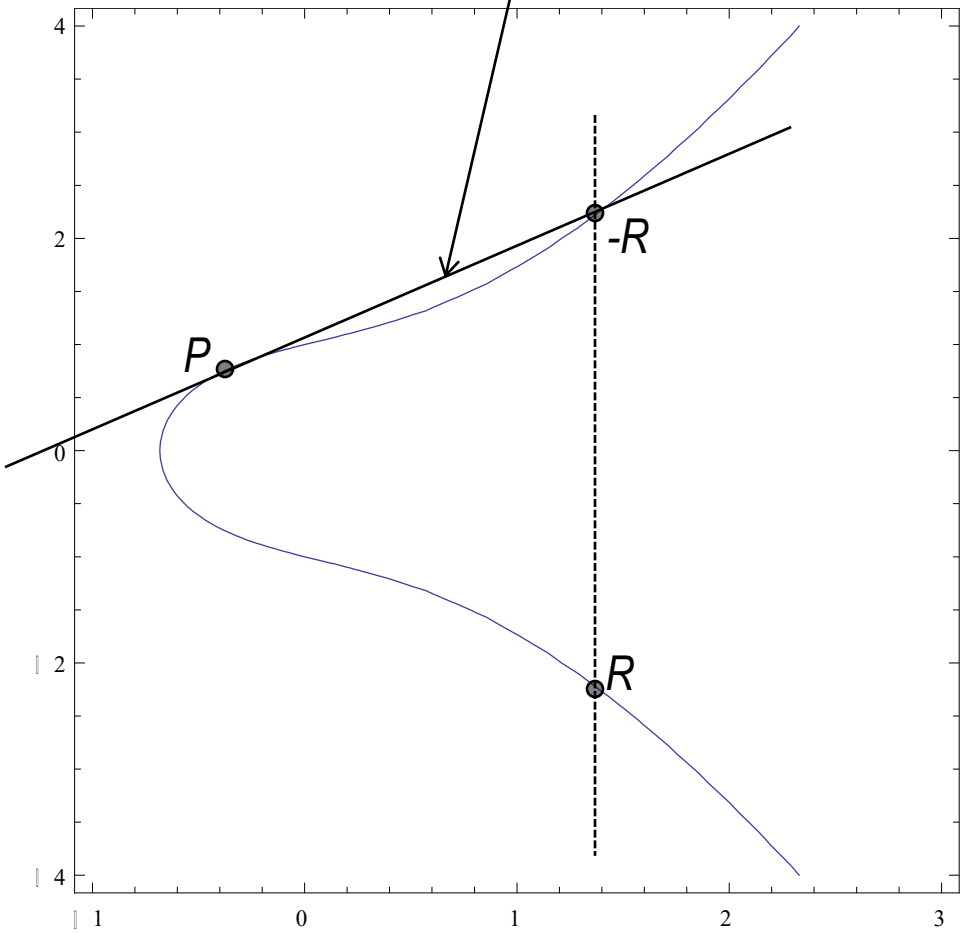
hence

$$\lambda^2 = x_1 + x_2 + x_3 \rightarrow x_3 = \lambda^2 - x_1 - x_2$$

y_3 now trivial from line eq.

P=Q sketch

$$y = y_1 + y'_1 (x - x_1) = \lambda(x - x_1) + y_1$$



$$y^2 = x^3 + ax + b$$

$$2y\frac{dy}{dx} = 3x^2 + a \rightarrow \frac{dy}{dx} = \frac{3x^2 + a}{2y}$$

$$\lambda = y'_1 = \frac{3x_1^2 + a}{2y_1}$$

Follow up as before...

Elliptic curve group
(over Zp)

EC over modular integers

- **Geometric interpretation**
  - *Over real numbers*
- **Our applications**
  - *Over modular integers*
- **Define "curve" as**
  - $E_p(x,y)$ *such that*
    - x,y integers in Zp
    - $y^2 \bmod p = x^3 + a\,x + b \bmod p$
- **Finite field → finite # of points**

Example: x^3+x+1 mod 5

➔ **All points:**

⇨ *pairs (i,j), i in [0,4], j in [0,4] s.t. $j^2=i^3+i+1$ mod 5*

⇨ *And O*

$$E(Z_5)=\{O,(0,1),(0,4),(2,1),(2,4),(3,1),(3,4),(4,2),(4,3)\}$$

⇨ *Result: 9 points*

⇨ *Let P=(0,1)*

➔ k P=P+P+…+P
generates all the points

$$P=(0,1)$$
$$2P=(0,1)+(0,1)=(4,2)$$
$$3P=(4,2)+(0,1)=(2,1)$$
$$4P=(2,1)+(0,1)=(3,4)$$
$$5P=(3,4)+(0,1)=(3,1)$$
$$6P=(3,1)+(0,1)=(2,4)$$
$$7P=(2,4)+(0,1)=(4,3)$$
$$8P=(4,3)+(0,1)=(0,4)$$
$$9P=(0,4)+(0,1)=O$$

example computation

$(0,1)+(0,1)$

step1 : compute λ

$$\lambda = \frac{3x_1^2 + a}{2y_1} = 1 \cdot 2^{-1} = 1 \cdot 3 = 3 \mod 5$$

step2 : compute x_3

$$x_3 = \lambda^2 - x_1 - x_1 = 3^2 = 4 \mod 5$$

step3 : compute y_3

$$y_3 = \lambda(x_1 - x_3) - y_1 = 3(0-4) - 1 = 2 \mod 5$$

result : $(4,2)$

$(4,2)+(0,1)$

step1 : compute λ

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} = 1 \cdot 4^{-1} = 4 \mod 5$$

step2 : compute x_3

$$x_3 = \lambda^2 - x_1 - x_1 = 2 \mod 5$$

step3 : compute y_3

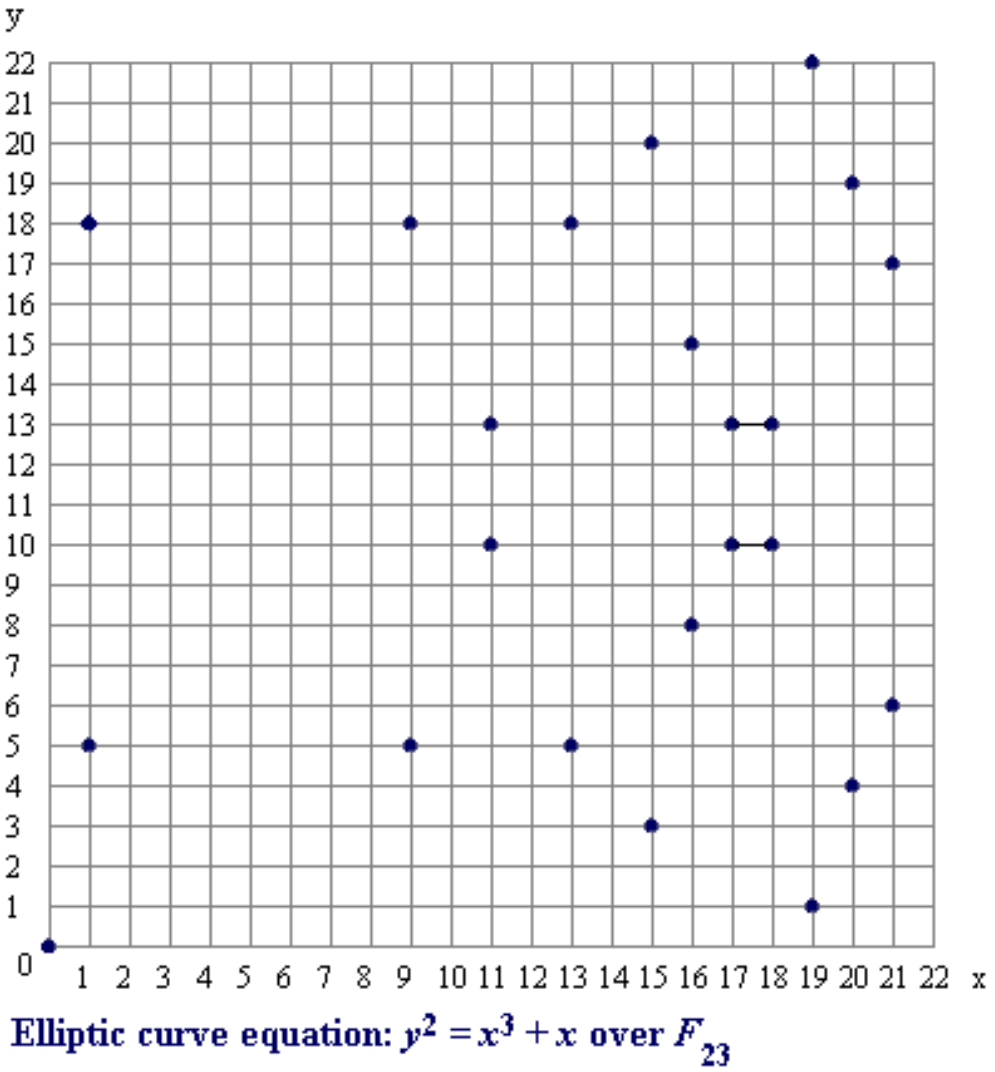
$$y_3 = \lambda(x_1 - x_3) - y_1 = 4(4-2) - 2 = 1 \mod 5$$

result : $(2,1)$

Many online calculators available e.g. https://graui.de/code/elliptic2/

Which points in “larger” group?

No geometric “look&feel” anymore…
no problem!



EC group

➔**$(E(Z_p),+)$ is a group**

1. *Addition* **closed** *on set E*
2. *Addition is* **commutative**
3. *O is* **identity** *wrt addition*
4. *Every point P in E has inverse –P wrt addition*
5. *Associative property holds*

Harder to prove in generality, but holds

Abelian group

EC Group properties for crypto

➔ **Easy/hard operation: [k]P**

- ⇨ *Given P and k, easy to compute [k]P*
- ⇨ *Given P and [k]P, hard to compute k*
  - →P = elliptic curve point on group
  - →k = integer mod group order
- ⇨ *"Equivalent" to Dlog problem on Zp*
  - →If we had used multiplicative notation: k → P^k

➔ **CRUCIAL: DEPEND ON THE CHOSEN CURVE!!!**

- ⇨ *Use curves recommended by standards!!!!*
- ⇨ *Curves not only in Zp, but also in GF(2^m) – Binary curves: convenient!*
- ⇨ *More recently: Edwards curves (different equation – no more Weierstrass)*

➔ **Different EC curves satisfy different assumptions!**

- ⇨ *We'll see (and exploit!) later on: e.g. CDH hard but DDH easy*

Multiplication (exponentiation)

- **Group order 160 bits → $q=2^{160}$**
- **Major cost: multiplication**
  - *analogous to exp*
  - *Uses very large factors k (160 bits)*
- **Efficiency: approx 1.5 Log_2 q**
  - *i.e. linear with the number of bits*
    - Basic algorithm: double and sum
      - » (analogous to square & multiply)

Example

- **Goal: compute [1437]P**
- **Express in bits**
  - $1437_{10}=10110011101_2$
- **Initialize first line**
  - *Differs if 0 or 1*
- **Start from 2° lsb to msb**
  - *For every bit*
    - Double;
    - If 1: add to result
- **Complexity:**
  - *O(nbit) x2*
  - *O(nbit/2) additions*

$lsb->msb$	*double*	*result*
1	$1P$	$+1P=P$
0	$2P$	.
1	$4P$	$+4P=5P$
1	$8P$	$+8P=13P$
1	$16P$	$+16P=29P$
0	$32P$	.
0	$64P$	.
1	$128P$	$+128P=157P$
1	$256P$	$+256P=413P$
0	$512P$	.
1	$1024P$	$+1024P=1437P$

No algorithm such as this for the opposite problem!! That's why it is hard!

Elliptic curve crypto
(examples)

EC crypto

- **Invented in 1985**
  - *Neal Koblitz*
  - *Victor Miller*
  - *(independently)*
- **Security: based on hardness of:**
  - *Given P and [k]P, hard to compute k*
  - *Elliptic Curve Discrete Log Problem – ECDLP*
- **Much shorter keys than RSA/DLP**
  - *And better scaling!*

Symm key equiv	*modulus size*	*Elliptic Curve*
80 bits	*1024 bits*	*163 bits*
128 bits	*3072 bits*	*283 bits*
256 bits	***15360** bits*	*571 bits*

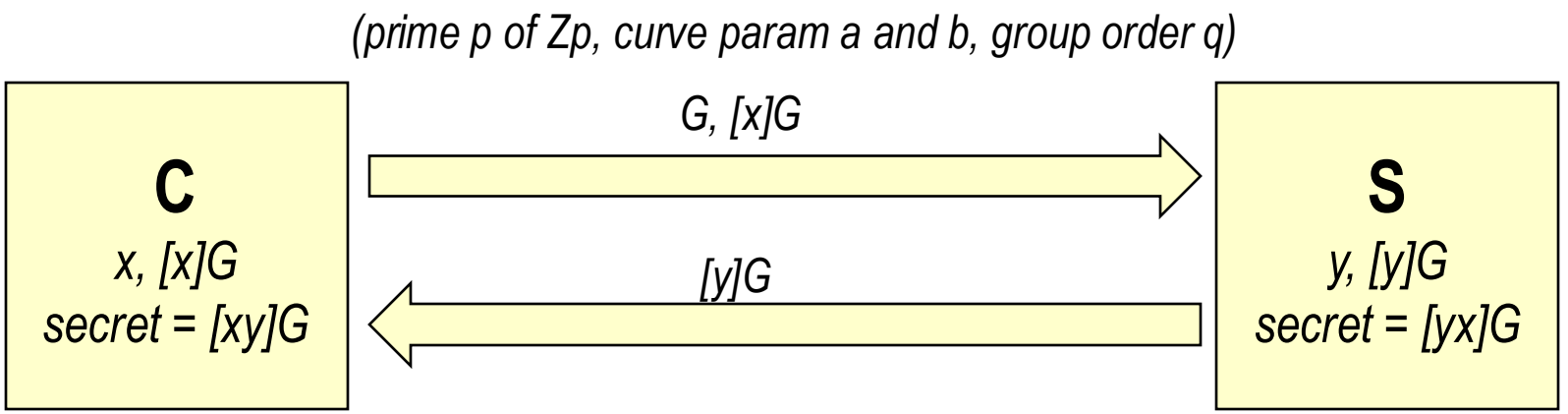
ECDH/ECDSA

- **Topmost used, since included in US National Security Agency – suite B ciphers**
- **Suite B: NIST-approved, promoted in 2005, adopted in TLS, IPsec, S/MIME, etc**
- **Algorithms:**
  - *Encryption: AES 128/256*
  - *Key Exchange: ECDH*
    - 256 and 384 bit prime curves
  - *Digital Signature: Elliptic Curve Digital Signature Algorithm (ECDSA)*
    - 256 and 384-bit prime moduli
  - *Hashing: SHA-256 and SHA-384*

ECDH

➔Exactly as DH

⇨*Just use as group an EC group!*



ECDSA

- **Elliptic Curve Digital Signature Algorithm**
  - *EC version of DSA*
    - Invented by Vanstone
    - Proposed by NIST in august 1991
  - *adopted by NSA Suite B*
- **Variant of ElGamal**
  - *Dlog based*
- **Famous also for other reasons ☺**
  - *Broken in Playstation 3, 2010*
    - Not because ECDSA is vulnerable…
    - Because of incorrect usage, see later

Let's first remember DSA
(simplified)

- **Multiplicative group**
  - *Z*(p), p>2 prime*
  - *Large prime q in group order*
    - Usually p = 2q+1 (safe prime)
- **Generator**
  - *g s.t. g^x mod p spans all group elements*
- **Private/Public keys**
  - *d, y = g^d mod p*

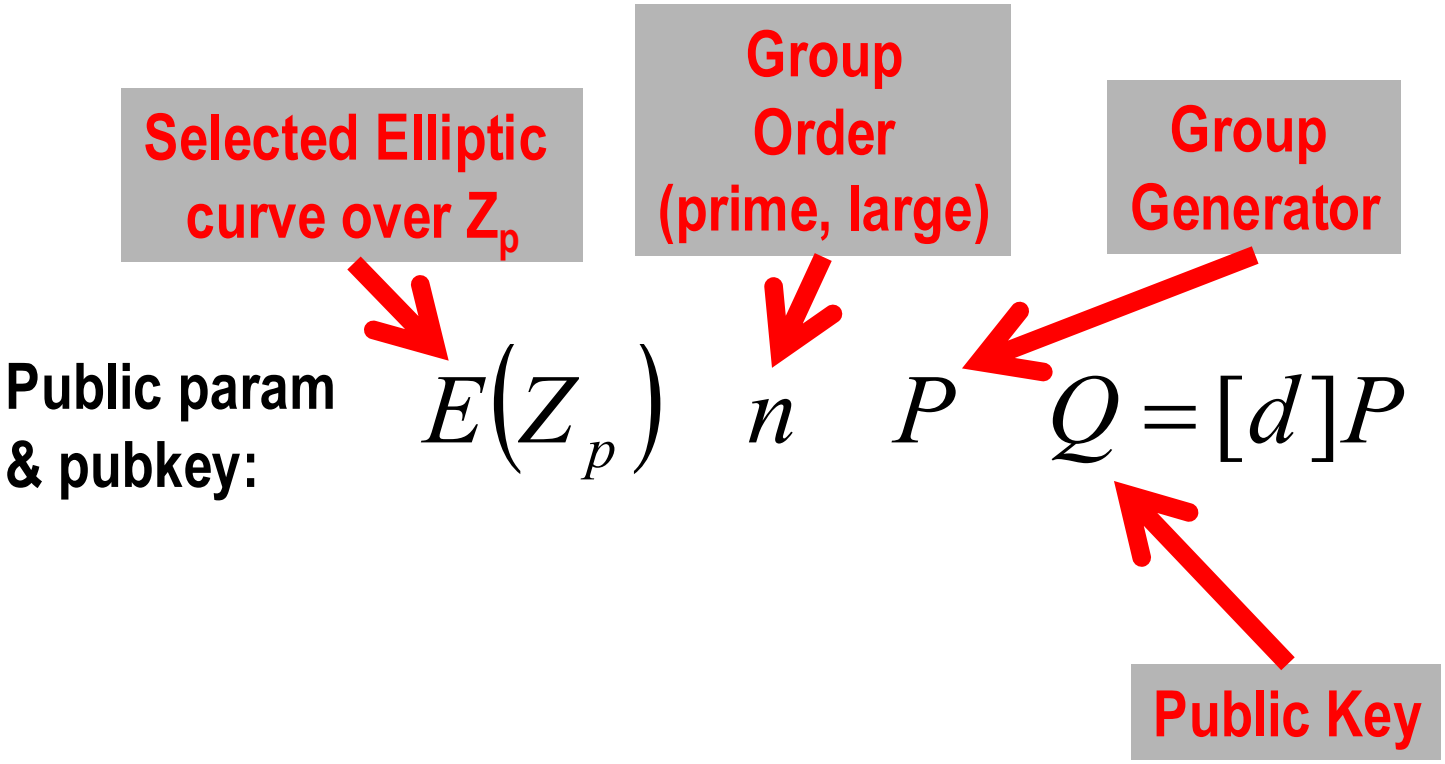
Let's first remember DSA
(simplified)

- **Signature for message m**
  - *Random k in (1,q)*
  - *Compute r = (g^k mod p) mod q*
  - *Compute s = k^{-1} (H(m)+ d r) mod q*
  - *Signature: (r,s)*
- **Verification**
  - *Compute u_1 = s^{-1} H(m) mod q*
  - *Compute u_2 = s^{-1} r mod q*
  - *Verify ((g^{u1} y^{u2}) mod p) mod q = r*

$$g^{u_1} y^{u_2} = g^{u_1} g^{d \cdot u_2}$$

$$u_1 + d \cdot u_2 = \frac{H(m)k}{H(m) + rd} + \frac{d \cdot r \cdot k}{H(m) + rd} = k$$

ECDSA: setup



➔**Private key:**

⇨*d = random integer in [1,n-1]*

⇨*Q = dP*

ECDSA: signature generation

➔ **k in [1,n-1]: random integer**

⇨ *ESSENTIAL: MUST BE*

→unique per each signature

→unpredictable

➔ **Compute kP=(x_1,y_1) EC point**

⇨ *x_1 is integer, then*

➔ **Compute r = x_1 mod n**

➔ **Compute k^{-1} mod n**

➔ **Hash function H(.) e.g. SHA**

Signature:

(r,s)

$$s=k^{-1}(H(m) + d\ r)$$

ECDSA: verification

- **From message m**
  - ⇨ *compute H(m)*
- **From s**
  - ⇨ *compute $w=s^{-1} \bmod n$*
- **Compute integers:**
  - ⇨ *$u_1 = H[m]\, w \bmod n$*
  - ⇨ *$u_2 = r\, w$*
- **Compute elliptic point**
  - ⇨ *$u_1 P + u_2 Q = (x_0, y_0)$*
- **Compute**
  - ⇨ *$v = x_0 \bmod n$*

Verification:

$v=r$

$$w = s^{-1} = k\left[H(m) + rd\right]^{-1}$$

$$u_1 P + u_2 Q =$$

$$= \frac{H(m)k}{H(m)+rd} P + \frac{r \cdot k}{H(m)+rd} Q =$$

$$= \frac{H(m)k}{H(m)+rd} P + \frac{rd \cdot k}{H(m)+rd} P = kP$$

Note: k never disclosed!

What if k predicted??

➔Private key would be disclosed!

from (r, s) and knowledge of k :

$$s = k^{-1}\left[H(m) + rd\right] \mod n$$

$$sk = H(m) + rd \mod n$$

$$\left[sk - H(m)\right]r^{-1} = d \mod n$$

d readily computed!!

What if k repeated??

➔Private key would be disclosed!

from $(r, s_1), (r, s_2)$ - same unknown k = same r

$$\begin{cases} s_1 = k^{-1}\left[H(m_1) + rd\right] \\ s_2 = k^{-1}\left[H(m_2) + rd\right] \end{cases} \mod n \quad \Rightarrow \quad \begin{cases} k = s_1^{-1}\left[H(m_1) + rd\right] \\ k = s_2^{-1}\left[H(m_2) + rd\right] \end{cases} \mod n$$

$$s_1^{-1}\left[H(m_1) + rd\right] = s_2^{-1}\left[H(m_2) + rd\right]$$

$$s_2 H(m_1) + s_2 rd = s_1 H(m_2) + s_1 rd$$

$$d(s_2 - s_1)r = s_1 H(m_2) - s_2 H(m_1)$$

$$d = \left[s_1 H(m_2) - s_2 H(m_1)\right] r^{-1} (s_2 - s_1)^{-1} \mod n$$

d readily computed!!

This is what happened with Sony's Playstation 3

see also https://bertcmiller.com/glimpse.html for another horror story